

Wave 7 — Production Readiness Plan

Date: 2026-07-16 **Baseline audited:** master c862dbb1a **Source:** six-agent production-readiness audit (secrets/config, auth, payments, ops, data/tests, storefront)

Locked decisions

Decision	Choice	Consequence
Launch market	US and EU	Sales tax and VAT/OSS required at checkout. Cookie consent + privacy policy are legal prerequisites, not hardening.
Deploy target	Single VPS, docker-compose	1 replica per service. Legitimises deferring ShedLock/leader-election and Eureka HA — but the constraint must be enforced, not assumed.
Payments	Stripe only	Wire the existing scaffold. Delete the dead PAYPAL/ALIPAY/WECHAT_PAY/APPLE_PAY/GOOGLE_PAY enum values rather than leave them as trust-the-client holes.
Scope	Minimum safe launch	Only what is unsafe, illegal, or impossible without. Everything else goes to the debt register at the bottom — explicitly, with the cut line written down.

Scope note: what "minimum" could NOT cut

Selling into the EU makes these launch-blocking regardless of scope preference:

- VAT calculation and collection at checkout (US sales tax likewise, via nexus rules).
- Cookie consent for Matomo — it is **live today with no banner**. DNT alone does not satisfy ePrivacy.
- A real privacy policy and terms of sale.

What minimum **did** cut, defensibly: **self-service** GDPR erasure/export. For launch these become a documented manual process behind a `privacy@` address with an SLA. Legal duty is discharged; the tooling is debt.

USER PREREQUISITES (block launch, not development)

Everything below must build, boot, and be verifiable **without** these. Same discipline as Wave 6: adapters disabled by default, failures → honest errors, never a 5xx.

1. **Stripe live account** + publishable/secret keys + webhook signing secret.

2. **Stripe Tax registration** for each US nexus state and the EU OSS scheme. *Without this, tax calculation cannot be correct and the store cannot legally sell.*
 3. **Domain + DNS** pointed at the VPS (needed for TLS and the Stripe webhook URL).
 4. **Real SMTP credentials** (Wave 6 shipped the outbox; it has never sent a real mail).
 5. **VPS** provisioned (suggest ≥ 8 GB RAM: 9 JVMs + MySQL + Elasticsearch is not small).
-

Cross-cutting landmines (apply to every block)

- **Flyway:** V42 is the last used (promotion's `social_post`). Wave-7 migrations claim **V43+** after checking `flyway_schema_history` immediately before first boot. `out-of-order: true` is permanent. Never `flyway repair`.
 - **litemall-db is shared and hand-maintained:** never regenerate; hand-edit entities + mapper XMLs together. After editing: `mvn install litemall-db`, restart every dependent, verify the nested `BOOT-INF/lib` copy in running exec jars.
 - **`andLogicalDeleted()` is inverted across 33 domain classes** (`Boolean.valueOf("0")` makes both enum constants `false`). Currently dormant — zero call sites. **Do not call it.** Bind the literal, as `LitemallAftersaleRepositoryImpl:45-60` already does.
 - **Config precedence trap:** core profile yml outranks service yml. Enable core-read config via ENV VARS.
 - **Never rebuild a jar under a running JVM** (hung statics). Kill first.
 - Verify live through the gateways (`:9000 / :9001` → `:8090` , `:18080`) — machine-token ~10-min TTL makes direct service curls flaky.
-

RESOLVED — guest cart (investigated 2026-07-16, post-plan)

Question: would requiring auth on `/srv/cart/**` break an anonymous add-to-cart funnel?

Answer: no. The edge-auth design is safe and cheaper than estimated. The guest cart is **client-side only** — held in `sessionStorage` and merged into the server cart on login (`cartSlice.ts:11-16`). `fetchCart` short-circuits before any server call when logged out (`cartSlice.ts:33-35`). Anonymous users never touch the server cart, so gating `/srv/cart/**` behind auth costs nothing. No `session_id` column, no merge path. **gateway-api Task A re-sized 2–5 days → 2–3 days, risk High → Low.**

But the investigation surfaced a P0 the audit missed — see `order` Task E0 below. It is now the highest-priority item in this plan, above the Stripe work.

● P0 DISCOVERED — client-controlled pricing (supersedes the IDOR finding)

The client sets the price of the order. Fixing Stripe does not fix this.

Verified chain:

1. `orderSlice.ts:158-168` — checkout mirrors each local cart line to the server, **sending price: it.price from client state**. This is the live production checkout path, not dead code.
2. `LiteMallCartController.toAggregate():122-124` — `cart.setPrice(new LiteMallMoney(req.getPrice()))`, straight from the request body.
3. `LiteMallCartServiceLayer.addCartItem():47-56` — persists it verbatim. **No re-resolution against the catalog.**
4. `LiteMallOrderDomainService.priceCalculation():18-29` — sums `checkedGoods.getPrice()` from those cart rows into `checkedGoodsPrice` → `orderTotalPrice` → `actualPrice`.

Impact: `POST /srv/cart/items {"goodsId":X,"price":0.01}` then submit → a real order for one cent. With Stripe correctly wired, the `PaymentIntent` is created for \$0.01 and **verifies successfully** — amount-matching (`order Task A1`) compares against an already-poisoned order total. This is orthogonal to the payment stub and survives every other fix in this plan.

Why it was missed: the payments audit checked `processPayment`, the storefront audit checked the Stripe stub, and neither traced the cart-write → order-total path across the module boundary. The IDOR finding pointed at the same endpoint but stopped at identity.

The correct pattern already exists in this codebase.

`LiteMallOrderOrchestratorService.addToCart():697-723` (backing the legacy `/srv/cart/add`) resolves goods+variant through `goodsFacade` and sets `cart.setPrice(product.getPrice())` — server-authoritative, client price ignored. The RESTful `/items` surface simply never adopted it.

Also found: `GET /srv/cart/items` is broken

`cartApi.list()` sends **no** `userId`, but the controller declares `@RequestParam Integer userId (required)` → **400 on every logged-in cart fetch**. `remoteAddToCartThunk` (the only caller of the body-`userId` POST outside the checkout mirror) is **never dispatched** — dead code. The cart page has been running on `sessionStorage` alone, silently swallowing the server error. Fix alongside E0; delete the dead thunk.

Worktree: `order` (litemall-order) — the money path

Branch: `fix/order` — FIRST: `git merge master`. **Scope:** `litemall-order/` + the migration it owns. Owns the money path end to end.

Task A — real Stripe payment verification (THE blocker)

Today `LitemallOrderOrchestratorService.processPayment()`:494-510 accepts any non-blank string as a confirmed `PaymentIntent`. Replace with:

1. **Server-side verification.** On pay, `PaymentIntent.retrieve(reference)`; assert **all** of: `status == "succeeded"`, `amount_received == order.actualPrice` (in minor units — no float), `currency` matches, and `metadata.orderId == order.id`. Any mismatch → reject, order stays `CREATED`, no CJ placement. Fail **closed**.
2. **Idempotency/replay.** Add a `UNIQUE` index on the payment reference. A `PaymentIntent` already bound to another order → reject. This is only possible once verification exists — today there is nothing to replay-check against.
3. **Webhook** `POST /srv/stripe/webhook/order-paid` — the configured `webhook-url` currently points at `example.com` and no controller exists. Must: verify the `Stripe-Signature` via `Webhook.constructEvent`, be **anonymous** in `svcsecurity` (Stripe cannot present a machine token) but signature-gated, be idempotent on `event.id` (persist seen ids), and handle `payment_intent.succeeded` / `.payment_failed`. The webhook — not the client call — is the authoritative paid signal; the client path is a latency optimisation.
4. **Delete the dead enums.** `PaymentMethod.PAYPAL/ALIPAY/WECHAT_PAY/APPLE_PAY/GOOGLE_PAY` have zero backing integration and route through the same trust-the-client path. Remove them; keep `CREDIT_CARD / DEBIT_CARD / WALLET`.
5. **Config:** `litemall.stripe.*` — `enabled:false` default, keys via ENV only, **no committed defaults** (follow `litemall-promotion-service/application.yml:58` — `#{MATOMO_AUTH_TOKEN:}` with no fallback). Disabled → card pay returns a clean typed error, never a fake success.

Task B — real refunds

`settleRefundToTender()`:1052-1058 only logs "would reverse". Implement `Refund.create` against the stored `PaymentIntent` for non-WALLET tenders; on API failure the order must **not** flip to `REFUNDED` — surface the failure to admin and leave it retryable. A refund that silently doesn't happen is worse than one that visibly fails.

Task C — tax (US sales tax + EU VAT)

1. Migration **V43+**: `tax_price decimal(10,2) NOT NULL DEFAULT 0.00` on `litemall_order`; consider `tax_breakdown` JSON for audit/invoices.

2. `TaxCalculationPort` in `application/ports` + `infrastructure/acl/stripe/StripeTaxAdapter` (domain never imports the client — module convention). Use `Stripe Tax Calculation` at quote time, referenced on the `PaymentIntent` at pay time.
3. **Fail closed.** Unlike Wave-6 social posting, a tax outage must block checkout, not ship untaxed. Disabled (`litemall.tax.enabled:false`) → checkout blocked with a clear error in any profile that has Stripe enabled.
4. Wire IOSS: existing `CjDropshipOrderFacadeImpl:54-61` IOSS declaration must stay consistent with VAT collected at checkout — if you collect EU VAT you declare IOSS, otherwise the buyer is charged twice at customs.

Task D — server-authoritative totals

`cartApi.ts:43` calls `GET /srv/cart/checkout`, **which does not exist**; the SPA falls back to a client-side total. With tax this becomes mandatory: implement it returning goods total, freight, tax, discounts, and grand total. The client must never compute money.

Task E0 — server-authoritative pricing (DO THIS FIRST — see the P0 section above)

Highest priority in the wave. Land before the Stripe work, so that amount-matching in Task A1 validates against a total the server actually owns.

1. **Cart-add must resolve price server-side.** `LitemallCartServiceLayer.addCartItem()` / the `/items` POST path must look up goods+variant via `goodsFacade` and set the price from `product.getPrice()`, exactly as `addToCart():697-723` already does. **Delete price from `AddCartItemRequest`** — don't merely ignore it; a field that looks authoritative and isn't will be re-trusted by the next person. Same for `goodsSn / goodsName / picUrl` (display fields, but they end up on `order_goods` rows via `LitemallOrderServiceImpl:410-416` and shouldn't be client-authored either).
2. **Belt and braces at submit:** re-validate `checkedGoodsPrice` against live catalog prices before creating the order; a mismatch is a clean 422 ("prices changed, review your cart"), never a silent recompute. This also handles the legitimate case of a price changing while the cart sat idle.
3. **SPA:** stop sending `price` in the mirror (`orderSlice.ts:158-168`); delete the never-dispatched `remoteAddToCartThunk`.
4. **Regression to watch:** flash-deal pricing. The deal price must resolve server-side from the deals tables — the goods-management deal work assumed the cart carried the price. Verify a deal-priced line still charges the deal price after the client can no longer assert it.

Task E — cart IDOR + broken list + order_sn correctness

1. `LitemallCartController.java:32,38,44,51,62,68` — take identity from `@RequestHeader("X-User-Id")`, never `@RequestParam/body`. The legacy `/srv/cart/*`

methods in the same file (:82-103) already do this correctly; match them. **Resolved: no guest-card complication** — see above.

2. **Fix the broken list:** `GET /srv/cart/items` currently 400s for every logged-in fetch (client sends no `userId`; the param is required). Removing the param in favour of the header fixes this as a side effect — verify the cart page then loads server lines rather than sessionStorage alone.
3. Migration: `UNIQUE` on `litemall_order.order_sn` + index on `litemall_order.user_id`; same for `litemall_cart.user_id`. `generateOrderSn():166-176` checks uniqueness **per user** while reconciliation is global — fix the scope and let the DB enforce it.
4. Move `cjFulfillmentService.placeForPaidOrder (:333)` **out** of the class-level `@Transactional` — a sync Feign call inside a money transaction holds row locks and a pooled connection for the length of a CJ round trip. Use the existing `AFTER_COMMIT` + outbox pattern already proven in this module.

Acceptance

- `mvn -q -o -pl litemall-order -am compile` clean; boots with Stripe disabled (card pay → typed error, **not** a fake success); migration applies.
 - Stripe test mode: real Elements payment → verified, order PAID once. **Replay the same PaymentIntent on a second order → rejected.** Tampered amount → rejected. Webhook with a bad signature → 400, order untouched. Webhook replay of the same `event.id` → no double-processing.
 - Refund a card order → visible in the Stripe dashboard; forced API failure → order does **not** show REFUNDED.
 - Tax: US and EU addresses each produce a correct, server-computed total; tax provider down → checkout blocked, no untaxed order.
 - **Adversarial (all four are exploitable today; each must be dead):**
 - `POST /srv/order/{id}/actions/pay` with `{"paymentIntentId":"x"}` → rejected.
 - `POST /srv/cart/items {"goodsId":X,"price":0.01}` → the persisted line carries the **catalog** price, not 0.01; the resulting order total charges full price.
 - Replay a used PaymentIntent on a second order → rejected.
 - Tampered PaymentIntent amount → rejected.
 - `GET /srv/cart/items` returns the user's server cart (today: 400).
 - A flash-deal line still charges the deal price with no client-asserted price.
 - Wave-5/6 wallet, brokerage, aftersale, CJ, mail regression green.
-

Worktree: gateway-api (litemall-gateway-api) — the edge

Branch: fix/gateway-api — FIRST: git merge master. **Scope:** litemall-gateway-api/ (edge + customer SPA). NO migration.

Task A — actually enforce authentication (THE other blocker)

SecurityConfig.java:29 is anyExchange().permitAll(); IdentityForwardingFilter:28-31 admits authorization "is deliberately not enforced here yet". Because MachineTokenRelayFilter:44-62 attaches a machine token to every lb:// route regardless of caller, downstream authenticated() passes for anonymous requests.

1. Enumerate public paths explicitly (catalog, search, product detail, /auth/**, health, the Stripe webhook) and require an authenticated customer for everything else (/srv/cart/**, /srv/order/**, /srv/user/**, checkout).
2. **Only relay the machine token for requests that carry a verified customer identity** (or are on the public list). The relay is what makes the missing gate exploitable — close both.
3. Regression: anonymous browsing, search, product detail, and the ?invite= + utm_* capture must all still work.

Task B — real Stripe Elements

orderSlice.ts:228 fabricates pi_stub_\${orderId}; Checkout.tsx:731 tells the customer card capture is a placeholder. Replace with Stripe Elements: create the PaymentIntent server-side, confirm client-side, send the real id. Delete the stub path and the disclosure copy. Publishable key via runtime config (reuse the /auth/site-config seam from Wave 6) — absent ⇒ card payment cleanly unavailable, not stubbed.

Task C — cookie consent (legal blocker)

Matomo is live (shared/tracking/matomo.ts) with **no consent UI** — it drops first-party cookies for anyone who hasn't set DNT. Add an opt-in banner: no tracker injection before affirmative consent, a persisted choice, a withdrawal path, and keep the existing DNT hard opt-out. Matomo's disableCookies / requireConsent API is the intended hook.

Task D — legal pages (legal blocker)

Layout.tsx:407-423 renders "Conditions of Use", "Privacy Notice", "Cookie Preferences", "Contact Us" — **all four route to /help or /service**. They are decorative. Add real routed pages: Terms of Sale, Privacy Policy (must name Matomo, Stripe, CJ, and Mautic as processors), Cookie Policy (wired to the Task-C preference UI), Returns/Refund policy. Content is a legal deliverable, not an engineering one — **flag if no counsel-reviewed copy exists**; shipping invented legal text is its own risk.

Task E — turn the mail on

`application.yml:114-118` — `reset-mail.enabled:false` default means `ResetPassword.tsx:13-17` **hides the forgot-password tab entirely**. A stock deploy locks out anyone who forgets their password. Enable in the prod profile (needs the SMTP prerequisite) and verify the tab appears and the token round-trips.

Acceptance

- Compile + SPA build clean.
 - **Adversarial:** anonymous `GET /srv/cart/items?userId=2` through `:9000` → 401. Authenticated as user 1 requesting user 2's cart → 401/403. Both are exploitable today.
 - Anonymous browse/search/product-detail still 200. `?invite=` + `utm_*` on a product URL → both captured, routing normal.
 - Real Elements payment completes; no `pi_stub` string remains in the bundle.
 - Fresh visitor → **no Matomo cookie and no tracker request before consent**; accept → tracking starts; withdraw → stops. DNT still hard-opts-out.
 - All four footer links reach real distinct pages.
 - Reset flow: tab visible, mail arrives, token round-trips.
-

Worktree: `platform` (NEW — infra/ops)

Branch: `fix/platform` — FIRST: `git merge master`. **Scope:** `docker/`, `docker-compose/`, `.github/workflows/`, `deploy/`, `per-service` `application-prod.yml` (**creates prod profiles only — does not touch default yml**, to avoid conflicting with the other two worktrees).

This worktree exists because **nothing currently deploys this application**.

Task A — containerize the real architecture

The only Dockerfile packages the **deprecated monolith** on EOL `openjdk:8-jre`. `docker-compose-recover.yml` references `litemall/*` images no script ever builds. `deploy/bin/deploy.sh` is `nohup java -jar` on a hardcoded `/home/ubuntu` path.

1. A JDK-21 Dockerfile per service (9), multi-stage, non-root, container-aware JVM flags (`-XX:MaxRAMPercentage`) — **not** the unbounded `java -jar` of `docker/litemall/Dockerfile:3`.
2. `docker-compose.prod.yml` : all 9 services + MySQL + ES + Kafka, with `restart: unless-stopped`, memory limits, dependency ordering, and **healthchecks** (none exist for any litemall service today).
3. **Delete or fix** `docker-compose-recover.yml` and `deploy/` — they describe an architecture that no longer exists and will mislead whoever deploys.

4. Fix `dc-local.sh:7` / `dc-cloudconfig.sh:7` — both reference `docker-compose.base.yml`, **which does not exist**. The documented local bring-up is broken.

Task B — prod profiles + secrets out of the repo

No `application-prod.yml` exists for **any** service; all ship `active: dev` with DEBUG logging.

1. A real prod profile per service: INFO logging, no dev seed, prod pool sizing.
2. Every secret via ENV with **no committed fallback**. Pattern to copy: `litemall-promotion-service/application.yml:58` (`${MATOMO_AUTH_TOKEN:}`). Covers: DB password (`Calliste_1006`, 6+ files), CJ API keys (**two different values committed — resolve the sprawl**), Tencent COS, WeChat, Stripe.
3. **Fail startup** in prod when a JWT signing key is absent, instead of `RsaKeys.from()` silently generating an ephemeral per-boot pair (`RsaKeys.java:49-63`; `JwtService.java:51-56` only WARNs). Silent ephemeral keys mean every restart invalidates all tokens.
4. Rotate the committed keys — both realms' RSA private keys are in git history (`gateway-api/application.yml:155-183`, `gateway-admin/bootstrap.yml:48-76`). *Repo is private, so this is hygiene, not incident response — but the admin signing key being readable by anyone with repo access is exactly the "leaked machine token" scenario.*
5. Replace the committed machine-client dev secrets (`gateway-api-dev-secret`, `AuthServerProps.java:16,19`).
6. **Lock down litemall-config**: `application.yml:3-11` exposes `include: "*" plus env.post.enabled: true with no Spring Security in the module — a writable, unauthenticated /actuator/env. Restrict to health,info, kill the POST, bind to the internal network.`
7. Untrack `backup/*.sql` and `litemall-all/backup/*.sql` (full dumps incl. the bcrypt admin hash), and add a generic `.env*` gitignore rule — only `docker-compose/.env.marketing` is covered today.
8. Point the config server away from the **public** `jovinuxien/litemall-config`, or leave it unused. It currently holds no secrets (all files 0 bytes bar a stale dev yml routing to Wave-4-deleted `wx-api/admin-api`), but its empty `application-prod.yml` files are a trap: filling them in publishes prod config.

Task C — TLS + ingress

No TLS config exists anywhere. The only nginx sample (`doc/conf/nginx.conf`) is a 2018 relic with deprecated TLSv1/1.1 proxying to the dead monolith. Add Caddy (or nginx + certbot) terminating TLS for both gateways, HSTS, HTTP→HTTPS redirect, and real CORS origins (currently hardcoded `http://localhost:9000`, `application.yml:22-24`). Stripe webhooks require a public HTTPS URL — this gates `order` Task A3.

Task D — CI that actually runs

`.github/workflows/main.yml` builds on JDK 8/11 (root pom needs **21**) and npm-tests `litemall-admin / litemall-vue`, **deleted in Wave 4** — nothing has been verified automatically in months.

1. Rewrite: JDK 21, build the real reactor, build both SPAs.
2. **Run the tests.** Root `pom.xml:45` sets `maven.test.skip=true` reactor-wide, and `litemall-db/pom.xml:218`'s `skipTests=false` does **not** override it (`maven.test.skip` is the master switch and wins). Fix the override so `litemall-order`'s 36 real tests and the Flyway/mapper tests run on every push. Fix or quarantine `MapperReturnTest`, which currently breaks test compilation.
3. Secret-scanning (gitleaks) on PRs so this class of finding doesn't recur.

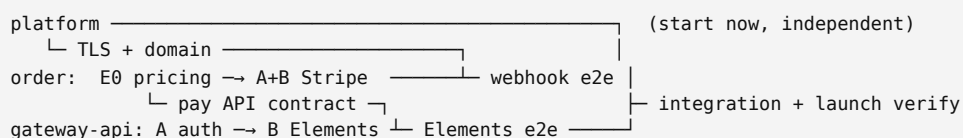
Task E — enforce the 1-replica constraint

Single-VPS makes the unguarded schedulers safe — **but nothing enforces it**. Every `@Scheduled` job lacks `ShedLock`/leader election: mail sweep, CJ status sync (per-replica external API budget), catalog refresh, `SocialDealAutoPoster` (would double-post to Meta/TikTok), brokerage unfreeze, outbox relay. Scaling any of these to 2 replicas causes duplicate customer emails and double social posts. **Pin replicas: 1 in compose with a comment pointing at this constraint, and add it to the debt register.** The trap is a future operator scaling out and silently double-charging the CJ API.

Acceptance

- `docker compose -f docker-compose.prod.yml up` brings all 9 services healthy from a clean checkout on a fresh host, with **zero secrets in the repo** — every credential from `env/` `.env` (gitignored).
- Boot with a required secret missing → **fails fast with a clear message**, does not silently generate an ephemeral key.
- `curl -X POST https://<host>/actuator/env` on the config server → 404/401, not 200.
- TLS: A-grade handshake on both gateways; HTTP redirects; Stripe reaches the webhook over public HTTPS.
- CI: green on push, **actually runs tests** (deliberately break one → CI red).
- gitleaks clean.

Sequencing



- **order Task E0 (server-authoritative pricing) is first, before Stripe.** Amount-matching is meaningless while the client still sets the total it's matched against.
- **platform starts now** — independent, and its TLS/domain work gates the Stripe webhook, so it is the critical path despite feeling like the least urgent.
- **order and gateway-api start together.** Contract to agree on day one: the PaymentIntent creation endpoint shape + the publishable-key config seam. **order** commits a `docs/handoff-stripe-checkout.md` the way Wave 6 did.
- **Do not merge order without gateway-api** . Verified payments behind an unauthenticated edge is still exploitable — and enforcing auth without real payments strands the funnel. They land together or not at all.
- **Final integration wave:** full e2e on the VPS with real Stripe test keys, a real domain, and MailHog→real SMTP swap.

Rough sizing (engineering days, excluding prerequisites and legal copy)

Block	Days	Risk
order E0 (server-authoritative pricing)	2–3	Medium — the pattern exists (<code>addToCart</code>); flash-deal pricing is the regression risk
order A+B (Stripe + refunds)	4–6	Medium — scaffold exists
order C (tax)	4–7	High — Stripe Tax registration is a hard external dep; correctness is legally consequential
order D+E (totals, IDOR, broken list, order_sn, tx)	3–4	Low–Medium
gateway-api A (enforce auth)	2–3	~~High~~ Low — guest cart is client-side only (resolved)
gateway-api B (Elements)	2–3	Low
gateway-api C+D+E (consent, legal, mail)	3–4	Medium — legal copy is not ours to write
platform A–E	6–9	Medium — mostly unglamorous, wide

≈ **26–39 engineering days** — E0 added ~2–3, the resolved guest-cart question gave ~2 back. Plus prerequisite lead time: Stripe Tax registration and counsel-reviewed legal copy both have real external latency — start them **today**, they are likely the true critical path.

Debt register — knowingly NOT in this wave

Accepted for a minimum-safe launch. Each is a real gap; none is unsafe or illegal at 1 replica with the above fixed.

Item	Why deferrable	When it bites
Flat admin RBAC (<code>AuthController.java:59,88</code> hardcodes <code>ROLE_ADMIN</code>)	Small trusted admin team at launch	First non-founder admin hire. A support agent can currently drain wallets.
No metrics/tracing/error tracking (no micrometer in any pom, no Sentry, 6/9 services lack logback config)	One host — <code>docker logs</code> is survivable	First production incident, which you will debug blind. Cheapest high-value follow-up.
ShedLock / leader election	Pinned to 1 replica (platform Task E)	The moment anyone scales out. Duplicate emails, double social posts.
Login rate limiting / lockout / CAPTCHA	No known abuse yet	Credential stuffing. Genuinely borderline — promote if launching loud.
<code>andLogicalDeleted()</code> inverted across 33 domains	Dormant — zero call sites	Whenever someone writes a new repository naively. Add a failing guard test cheaply.
GDPR self-service erasure/export	Manual process behind <code>privacy@</code> + SLA discharges the duty	Volume, or a regulator asking for the process.
SEO (CSR-only, one static <code><title></code> , no sitemap/meta/structured data)	Not a safety issue	Organic growth. Large latent cost for a store, and it compounds.
Three divergent schema copies (<code>litemall-db/sql/README.md</code> — untranslated — tells deployers to use a dump 41 migrations stale)	Flyway is authoritative and works	A deployer follows the README. Delete or clearly mark the stale dumps in this wave — it's a 5-minute fix.
8,450 residual Chinese strings in <code>litemall_data.sql</code> ; 7 live untranslated demo reviews; seeded <code>admin123/mall123/user123</code> with no prod gate	Prod won't load the demo seed	Anyone loading the seed into prod. Gate it.
Eureka + config server SPOFs	Same host as everything else	Multi-host.
ES single-node with an open JDWP debug agent (<code>docker-compose.yml:48-64</code> , binds <code>*:8000</code>)	Prod compose won't inherit it	Only if that compose file is reused. Strip the agent regardless.

Verification philosophy for this wave

Wave 6 verified features by driving them. This wave is different: **the deliverables are absences of vulnerabilities**, and absence doesn't demo. Every security acceptance above is written as an **adversarial** check — send the exploit, prove it fails. Specifically:

- `POST /srv/cart/items {"price":0.01}` → line persists at the catalog price. **(The one that survives every other fix.)**
- `POST /srv/order/{id}/actions/pay` with a junk `paymentIntentId` → rejected.
- Anonymous `GET /srv/cart/items?userId=<victim>` → 401.
- Replay a used `PaymentIntent` on a second order → rejected.
- Webhook with a forged signature → 400.

These five are exploitable today. They are the wave's real definition of done.

A lesson worth carrying: the price-tampering P0 sat in the seam between two audits — one owned `processPayment`, the other owned the SPA's Stripe stub, and the vulnerability lived in the cart-write → order-total path that crossed both. The six-agent audit found ~40 real issues and still missed the single worst one, because each agent respected a module boundary the exploit did not. **Trace money end-to-end, across module boundaries, as its own exercise** — the price a customer pays is assembled from four files in three packages, and no single-module review will ever see it whole.